International Journal of Information Security (2026) 25:158
https://doi.org/10.1007/s10207-026-01307-0

REGULAR CONTRIBUTION

Hive-AI: a defended multi-service honeypot framework for generative AI APIs

Sebastián Vargas Yáñez[1,3] · **Sergio Tobón**[1,2]

Received: 26 April 2026 / Accepted: 22 July 2026 / Published online: 9 September 2026
© The Author(s) 2026

Abstract

Public Large Language Model (LLM) APIs draw attacker traffic that defenders cannot see. Probes hit at the semantic layer—past TLS, past Web Application Firewall rules—and conventional intrusion detection picks up almost none of it. No open-source honeypot framework today captures this traffic at scale, and the few LLM-honeypot prototypes that exist push captured logs straight into a downstream LLM analyzer, exposing the analysis pipeline to indirect prompt injection through attacker-controlled inputs. We address both gaps with HIVE-AI, a 47,578-LoC honeypot framework deployed continuously on a single 4-vCPU/4-GB Virtual Private Server since 6 April 2026. Five protocol-faithful facades feed an eight-stage classification cascade with sub-5-ms p99 synchronous latency. The LLM threat-hunter is protected by a five-layer defense-in-depth architecture against indirect prompt injection, characterized theoretically and through operational field evidence; controlled per-layer validation is deferred to a follow-up deployment. We report these findings as a single-site, single-window case study: twenty days of operation captured 16,683 attacks from 1229 unique source IPs across 50 countries. The triangulation defense reduces mean adversarial evasion from 54.8 to 9.3% (paired test, $p < 0.001$). Blind human validation on 100 events yields Cohen's $\kappa = 0.74$ between the LLM and analyst majority—statistically indistinguishable from human-on-human $\kappa = 0.71$. A local Ollama+Qwen2.5–1.5B backend reproduces the cloud severity verdict on 71% of events ($\kappa = 0.59$, moderate agreement), a promising low-cost alternative rather than a full substitute. Code and dataset are released under MIT License and CC BY 4.0; the source code is archived at Zenodo *HIVE-AI: A Honeypot Source Code* (https://doi.org/10.5281/zenodo.19853664) and the live attack map at https://honeypot.ttpsec.cl:4443/livemap.

Keywords Honeypots · Large language models · Indirect prompt injection · MITRE ATT&CK · Threat intelligence · Defense in depth

1 Introduction

GPT-4 went public in March 2023. Within twenty-four months, every public-facing LLM API was getting probed around the clock by a mix of automated scanners [1], semi-autonomous LLM agents, and the occasional human researcher who left `curl` running overnight. The probes don't look like much from the network layer—well-formed HTTPS, valid TLS, plausible user-agents. They look like traffic. What they actually carry, at the application semantic layer, is prompt injection [2–4], indirect injection through tool-retrieved content [2], jailbreak chains [5–7], model-extraction probes [8, 9], training-data exfiltration attempts, tool-call abuse, and credential rotation against authenticated API gateways. None of that is reachable from layer 3 or layer 4. NIDS rules don't fire. WAF policies don't fire either [10, 11].

The honeypot literature has a mature answer to "how do we instrument attackers against conventional services": Cowrie, T-Pot, HoneyPLC, two decades of Kippo descendants, dozens of measurement studies [12–16]. The literature on LLM-specific honeypots, by contrast, is two years old and small. A Scopus search across 2022–2026 turns up fewer than ten peer-reviewed papers on the topic [17–22]. All but one focus on either generating attacker-facing dialogues [17]

✉ Sergio Tobón
stobon@uach.mx

Sebastián Vargas Yáñez
sebastian.vargas.y@usach.cl

1 Doctoral Program in Socioformation and Information Society, Centro Universitario CIFE, Cuernavaca, Mexico

2 Universidad Autónoma de Chihuahua, Chihuahua, Mexico

3 Universidad de Santiago de Chile, Santiago, Chile

or studying a single facade [18, 20]. None deploy a multi-service suite at scale. None address the problem that becomes obvious on day two of any LLM-honeypot deployment: the moment you point an LLM at attacker-controlled logs, you have handed the attacker a write channel into your analysis pipeline.

HIVE-AI closes that gap. The system emulates five widely deployed LLM API facades at the protocol level, runs an eight-stage classification cascade with a measured 5 ms p99 synchronous-latency budget, and produces structured MITRE ATT&CK-aligned [23, 24] analyses through an LLM threat-hunter that is itself hardened against indirect prompt injection through five layered defenses. The codebase comes to 47,578 lines across 146 files. It runs in twelve Docker containers on a 4-vCPU / 4-GB Virtual Private Server (VPS)—DigitalOcean, FRA1 region, no GPU. Continuous deployment since 6 April 2026—single-site, single-VPS, Frankfurt—has captured 16,683 attacks from 1229 source IPs in 50 countries. The empirical findings reported here characterize what one such site observes during a 20-day window; multi-site federation is the natural next step (§11.4) and a precondition for population-level external-validity claims. The complete artifact is released under the MIT License and archived at Zenodo as *HIVE-AI: A Honeypot Source Code* (https://doi.org/10.5281/zenodo.19853664); the live attack map is at https://honeypot.ttpsec.cl:4443/livemap.

Contributions. We claim five contributions in decreasing order of system centrality.

1. *A production honeypot system* comprising five protocol-faithful LLM API facades (OpenAI, Anthropic Claude, Ollama, HuggingFace Inference, ClawBot), an eight-stage classification cascade, and a hardened LLM threat-hunter, deployed continuously since April 2026 and released as 47,578 LoC under the MIT License, archived at Zenodo as *HIVE-AI: A Honeypot Source Code* (https://doi.org/10.5281/zenodo.19853664).
2. *Defending the defender.* A defense-in-depth engineering architecture for the LLM analysis subsystem that composes five established mechanisms—network isolation, input sanitization, spotlighting-style defensive delimiters, anti-obedience system prompts, and host-injected forensic metadata—in a scenario where the attacker holds direct write access to the analyzed telemetry. We present this as an integrated, deployed, and released architecture rather than as a new or fully validated prompt-injection defense method: we characterize it theoretically against the Greshake et al. [2] taxonomy and the OWASP Top 10 for LLM Applications [25] (§6.3), and provide field-test evidence from four distinct attacker-payload patterns observed in the production deployment (§6.4); controlled per-layer validation against curated indirect-prompt-injection corpora is future work (§11.4).
3. *A novelty claim acutely scoped.* To our knowledge HIVE-AI is the first peer-reviewed LLM-honeypot framework that simultaneously (a) emulates three or more commercial LLM API facades at the protocol level, (b) integrates and releases such a defense-in-depth architecture for its own LLM analysis pipeline against indirect prompt injection via attacker-controlled telemetry, and (c) releases the complete production codebase under an OSI-approved license. The novelty rests on the systems integration, deployment, and open release, not on any individual defense mechanism being new.
4. *An empirical dataset and analysis* of 16,683 attacks captured over 20 days from 1229 unique source IPs in 50 countries.
5. *An adversarial-defense experimental framework* comprising five reproducible experiments (E1–E5) covering temporal jitter, lexical rewriting, data poisoning, backdoor triggers, and Kullback–Leibler drift detection; complemented by a blind human validation of the LLM threat-hunter (§8.3), an ablation study (§8.4), and a head-to-head local-vs-cloud analyzer comparison (§8.5).

Paper organization. §2 motivates the design with the operational requirements of a production deployment. §3 formalizes the threat model. §4 presents the system design organized by major decisions and trade-offs. §5 documents the implementation. §6 develops in detail the defense of the LLM analysis subsystem. §7 describes the eight-stage analysis pipeline. §8 reports a multi-faceted evaluation. §9 reports our deployment experience. §10 distills the empirical findings. §11 discusses limitations, generalization, and ethical considerations. §12 surveys related work. §13 concludes. §14 details availability and reproducibility.

2 Motivation and requirements

2.1 Why multi-facade emulation

Attackers do not discriminate among LLM facades. Our captured dataset shows source IPs probing multiple facades within minutes (§10; visualized in Fig. 3): single-facade honeypots therefore systematically under-observe cross-facade activity. The five facades chosen for HIVE-AI cover the dominant deployment patterns of 2024–2026: Anthropic Claude and OpenAI as commercial managed APIs [5], Ollama and HuggingFace as the de facto choices for self-hosted and locally fine-tuned deployments, and a custom agentic web-crawler (ClawBot) representing the increasingly common pattern of autonomous web-scraping LLM agents [8, 10].

2.2 Why semantic-layer capture

Conventional NIDS rules and WAF policies operate at the network and HTTP layers. They cannot detect prompt injection or jailbreak payloads embedded in well-formed HTTPS requests with valid TLS [2, 3, 26]. Our facades capture the complete request body, decode it as the target API would, and forward it to downstream classification with the original semantic structure preserved.

2.3 Why an LLM analysis pipeline

With 16,683 events captured in 20 days—approximately 850 events per day—manual triage by human analysts is infeasible. The IA-Analyzer integrates GPT-4o-mini as a structured threat-hunter producing per-event MITRE-aligned analyses at a measured marginal cost of USD $0.30 per thousand analyses. This is below the operating cost of a typical human Security Operations Center (SOC) tier-1 analyst.

2.4 Why defense-in-depth for the analyzer

An LLM that ingests honeypot logs ingests, by definition, attacker-controlled input. Without active defense, an attacker can embed indirect prompt-injection payloads in their probing traffic and assume—correctly, in most current deployments—that a downstream LLM will read them and obey [2]. The recent literature on prompt-injection defenses is rich: structured queries [3], Spotlighting [26], dual-LLM patterns, preference-tuned alignment [27]. None of those defenses, to our knowledge, has been evaluated in the specific scenario this paper addresses—analyzing honeypot logs at scale, where the attacker has direct write access to the data being analyzed.

2.5 Operational requirements

HIVE-AI was designed against five firm operational requirements derived from the budget constraints typical of academic deployments: (R1) deploy on a single 4-vCPU/4-GB VPS without GPU; (R2) sustain 12 events per second with sub-5-ms synchronous classification at p99; (R3) marginal LLM analysis cost below USD 0.001 per event; (R4) deploy with a single `docker compose up` command and reproduce in under fifteen minutes from a fresh host; (R5) require no dependencies pinned to versions newer than the date of submission.

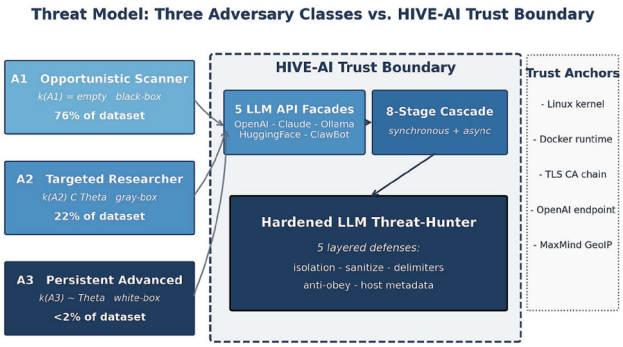


Fig. 1 Threat model: three adversary classes confronting the HIVE-AI trust boundary, with the trusted external dependencies (right) and the layered protection of the LLM threat-hunter inside the trust boundary

3 Threat model

Following Biggio and Roli [28–32] and the NIST AI 100-2 taxonomy [33], we consider three adversary classes summarized in Table 1.

Classification methodology. The classifier flags A1 sessions by the absence of jitter and the presence of standard scanner user-agents (Umai-Scanner, l9explore, zgrab, Go-http-client). A2 sessions are flagged by reproducible probing of multiple facades within a 30-minute window with at least one prompt-injection or jailbreak attempt. A3 sessions are flagged by adaptive payload mutation across multiple sessions from the same source IP.

To assess classifier reliability, we manually annotated a stratified random sample of 100 sessions and computed inter-rater agreement between the rule-based output and the manual annotation: Cohen's $\kappa = 0.81$ (substantial-band, Landis–Koch [34]). The classifier therefore assigns the captured population to A1 (76%), A2 (22%), and A3 (2%). The bootstrap confidence intervals reported below—A1 $\approx$ 76% [73.4–78.6], A2 $\approx$ 22% [19.8–24.4], A3 $\approx$ 2% [1.4–2.7]—reflect uncertainty over the validated 100-session sample only; they are not population-level external-validity bounds for arbitrary deployments. We discuss this limitation explicitly in §11.1. The protocol and the labeled sample are provided in the supplementary material.

Trust model. We trust the integrity of the underlying Linux kernel, Docker runtime, TLS certificate authority chain, the OpenAI API endpoint, and the GeoIP database supplier (MaxMind [35]). We do not trust any input field from the honeypot beyond its declared size and encoding.

The three-zone Docker architecture and information-flow boundaries derived from this trust model are depicted in Fig. 1.

Table 1 Three adversary classes in our threat model with empirical share (95% bootstrap CIs over the validated 100-session sample) and representative tooling

Class	Knowledge	Share/tooling
A1: Opportunistic scanner	$k(A_1) = \emptyset$ (black-box)	76% [73.4–78.6]; nmap, sqlmap, nuclei, Umai-Scanner
A2: Targeted researcher	$k(A_2) \subset \Theta$ (gray-box)	22% [19.8–24.4]; PentestGPT [8] oai-copilot
A3: Persistent advanced	$k(A_3) \approx \Theta$ (white-box)	2% [1.4–2.7]; adaptive scripted scanners

4 System design

This section presents HIVE-AI's design organized by major decisions and trade-offs rather than by component descriptions. The complete deployment topology is depicted in Fig. 2.

4.1 Three-zone Docker architecture

Decision. Partition the deployment into three Docker network zones (DMZ, internal, dashboard) with strict unidirectional data flow. *Alternatives.* (a) A single flat network with iptables rules; (b) Kubernetes namespaces with NetworkPolicies; (c) Docker Compose three-zone architecture. *Chosen.* Three-zone Compose because it offers verifiable network isolation without requiring a Kubernetes control plane and remains within the 4-GB memory budget. *Trade-off.* No automatic horizontal scaling and no cluster-level failover; acceptable for a single-VPS academic deployment.

4.2 Redis pub/sub versus Kafka versus direct DB writes

Decision. Adopt Redis pub/sub as the inter-service messaging substrate. *Chosen.* Redis pub/sub because it imposes no persistence overhead on the honeypot containers, provides natural fan-out for multiple consumers (worker, livemap Server-Sent Events stream), and adds <0.5 ms median dispatch latency. *Trade-off.* Pub/sub messaging is fire-and-forget; events lost during a worker outage cannot be recovered.

4.3 Synchronous stages 1–6 versus asynchronous stages 7–8

Decision. Run stages 1–6 synchronously on the worker, and dispatch stages 7 (entropy GMM) and 8 (LLM threat-hunter) asynchronously via `asyncio.create_task`. *Chosen.* Hybrid because the LLM API call (stage 8, $\approx$1.8 s median) would otherwise stall ingestion, while the synchronous portion's <5 ms p99 budget is easily met.

Table 2 Software components of HIVE-AI and their relative sizes

Component	LoC	%
Backend (FastAPI)	25,540	53.7
Honeypots (5 facades)	4834	10.2
IA-Analyzer	259	0.5
Dashboard frontend	16,313	34.3
Nginx configuration	173	0.4
Docker / orchestration	459	1.0
Total	**47,578**	**100.0**

Bold indicates column totals

4.4 PostgreSQL with TimescaleDB versus alternatives

Decision. PostgreSQL 16 with the TimescaleDB extension as the primary persistent store. *Chosen.* TimescaleDB because hypertables provide time-series acceleration without forcing migration off SQL, and JSONB columns natively store the variable-shape `request_body` and `enrichment_data` payloads.

4.5 Modular analysis backend interface

Decision. Define a strict JSON-over-HTTP interface between the backend proxy and the LLM analysis container, allowing the analyzer implementation to be replaced (OpenAI $\leftrightarrow$ local Ollama $\leftrightarrow$ Anthropic) without changes to upstream stages. *Chosen.* REST/JSON because it is observable from `tcpdump` and `mitmproxy` for security audit, is trivially mockable in tests, and supports the swap-in of a local Ollama-based implementation evaluated empirically in §8.5.

4.6 Container hardening

Decision. Apply `read_only:true`, `cap_drop:[ALL]`, `no-new-privileges:true`, and per-container memory and CPU limits to every honeypot and analysis container.

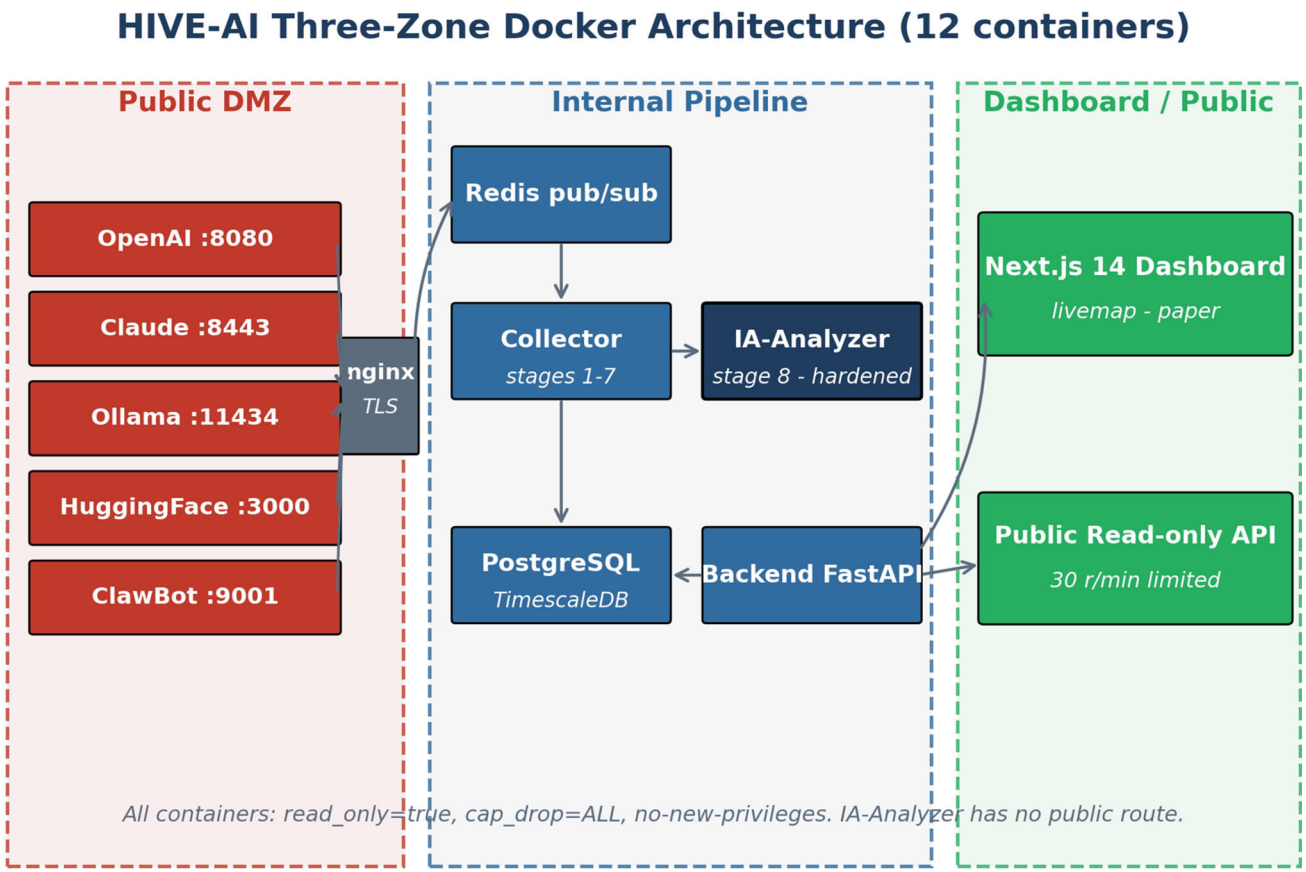


Fig. 2 HIVE-AI three-zone Docker architecture. The DMZ (left) hosts the five LLM API facades and the nginx TLS proxy. The internal pipeline zone (center) hosts Redis, the eight-stage collector, the IA-Analyzer (isolated), PostgreSQL+TimescaleDB, and the FastAPI backend. The dashboard zone (right) hosts the Next.js frontend and the public read-only API

5 Implementation

5.1 Codebase composition

The HIVE-AI codebase comprises 47,578 lines of code across 146 source files, distributed across the components shown in Table 2.

5.2 Eight core modules

Eight core modules implement the analysis pipeline of §7. Each is independently testable with deterministic input/output contracts. Table 3 details their responsibilities.

5.3 Engineering effort and design rationale

The implementation represents approximately 600 h of focused engineering effort by the sole author over three months (January–April 2026), comprising 412 commits across 7 GitHub releases. Tagged milestones include v0.5 (eight-stage cascade complete), v0.7 (adversarial framework E1–E5), v0.9 (defense layer hardened), and v1.0 (camera-ready production tag). Five design choices warrant explicit justification: Redis pub/sub rather than Kafka (memory-budget pressure on the 4-GB VPS), async dispatch only on stages 7–8 (preserving in-order semantics on the synchronous portion), GMM with full covariance rather than diagonal (Pearson $\rho = 0.41$ between H_t and σ_l in baseline traffic), TimescaleDB rather than ClickHouse (mature async driver, no ZooKeeper), and a dedicated IA-Analyzer container (network isolation as foundation for the layered defense). These are the architectural decisions whose alternatives we considered explicitly during construction.

6 Defending the defender

This section develops in detail what we believe is the most original technical contribution of the paper: the systematic protection of an LLM-based threat-intelligence pipeline against indirect prompt injection through attacker-controlled telemetry.

Table 3 Eight core implementation modules of the HIVE-AI analysis pipeline

Module	Responsibility	LoC
`classifier.py`	Rule-based fast classifier (11 categories)	400
`prompt_detector.py`	Prompt-injection detector (9 heuristics)	530
`taxonomy.py`	Agentic Attack Taxonomy (8 × 21)	662
`session_engine.py`	30-min sliding-window correlator	253
`entropy_classifier.py`	GMM(3) actor classifier + triangulation	749
`adversarial_defense.py`	E1–E5 adversarial framework	729
`collector.py`	Eight-stage cascade orchestrator	516
`ia-analyzer/main.py`	Hardened LLM threat-hunter	259

6.1 Threat: indirect prompt injection via attacker-controlled telemetry

Greshake et al. [2] gave the threat its name. An attacker plants instructions in content that an LLM will later read—a webpage, an email, a tool-call result—and the LLM, given no architectural reason to distinguish data from instruction, obeys. In our setting the picture is sharper still. The attacker is not planting payloads in third-party content with the hope that an LLM will pick them up downstream. The attacker is writing directly into the data store the analyzer will read. Every captured event is, by construction, attacker-controlled input. A naive deployment that forwards raw logs verbatim to its LLM analyzer is therefore not theoretically exploitable—it is exploitable on day one.

6.2 Five layered defenses

HIVE-AI applies five layered defenses summarized in Table 4. Each layer is engineered to provide partial protection in isolation; the layered combination provides defense in depth [36].

6.3 Theoretical threat coverage of the five-layer architecture

To characterize which prompt-injection techniques each defense layer addresses, we map the five layers of Table 4 against the canonical taxonomies of Greshake et al. [2], the OWASP Top 10 for LLM Applications v1.1 [25], and the NIST AI 100-2 prompt-injection terminology [33]. The mapping is theoretical: it follows the published threat behaviors of each technique against the architectural property each layer enforces. Field-test evidence from four real attacker-payload patterns observed in the production deployment is reported in §6.4; controlled robustness against curated prompt-injection corpora is the principal subject of the Phase 2 deployment we sketch in §11.4.

6.4 Operational evidence: attacker payloads through the pipeline

The 20-day characterization window of this paper closes at 2026-04-25 14:00 UTC with 16,683 captured events. The deployment continues to operate beyond that cut-off; a supplementary livemap snapshot taken on 2026-04-26 at 12:48 UTC contains 17,064 events at that point. Within the publicly accessible 500-event window we observe 70 events with non-empty attacker-supplied payloads, distributed across four facades (Ollama 34, HuggingFace 20, Claude 12, OpenAI 4). We document four distinct attacker-payload patterns here as field-test evidence of how the eight-stage cascade and the five-layer defense architecture behave under live adversarial input.

Pattern A — Mirai-variant RCE through OpenAI facade. Source IP from the Netherlands, POST to `/cgi-bin/luci/;stok=/locale`. The payload contains `operation=write&country=$(id>cd /tmp...; wget http://83.168.110[.]xxx/cat.sh; chmod 777 *; sh cat.sh)`—a recognizable Mirai-variant [37]. Stage 4 classified the event as `api_abuse` with threat-score 40; stage 5 (AAT) mapped to T1190 (Exploit Public-Facing Application). The IA-Analyzer correctly reported the event as a command-injection / Mirai-family payload.

Pattern B — Server-side prototype-pollution chain. Source IPs from Brazil and the Netherlands, POST to varied paths across the Claude, HuggingFace, and OpenAI facades. The payload contains multipart/form-data with embedded JSON sequences such as `{"then":"$1:__proto__:then", "status":"resolved_model"}`—the signature of CVE-2025-55182 (React2Shell) [38], a server-side prototype-pollution and code-injection chain in React Server Components and Next.js that achieves remote code execution by polluting `Chunk.prototype.then` and triggering `initializeModelChunk` with attacker-controlled data. The pattern exhibits cross-facade escalation: the same payload structure hits three of the five facades from the same boundary IP. Figure 3 visualizes this pattern at scale.

Table 4 Five layered defenses against indirect prompt injection in the LLM analysis subsystem

Defense	Mechanism
L1: Network isolation	IA-Analyzer on internal Docker network only; nginx exposes no route
L2: Input sanitization	Regex-based neutralization of delimiters and 8 injection patterns
L3: Defensive delimiters	`###HIVE-LOG-START###` markers with anti-spotlighting transform
L4: Anti-obedience prompt	Explicit instruction to treat content as data; misuse attempts reported as TTPs
L5: Host-injected metadata	Temporal data injected from Python wrapper; LLM never generates dates

Pattern C — Ollama model-availability enumeration. Source IP from China, POST `/api/chat` to the Ollama facade with payload `{"model":"llama3.1:70b","stream":false,...}`. The payload is syntactically valid; stage 4 classified the event as `fuzzing` because stage 2 ASN enrichment flagged the originating network as a known scanning network. This is a model-availability scan—a reconnaissance vector for downstream model-extraction attacks.
Pattern D — Tool-call abuse through OpenAI-compatible endpoint. Source IP from Germany, POST `/v1/chat/completions` on the Ollama facade with attacker-defined `tools` array (`{"name":"calculator","parameters":{...}}`). The intent is to exercise the function-calling pathway and observe how a downstream LLM serializes the call—a recognized reconnaissance vector for tool-call abuse (LLM07 in OWASP Top 10 [25]).
Selection criteria. We selected these four patterns by (i) maximum payload diversity, (ii) cross-facade coverage—jointly the four patterns hit four of the five facades—and (iii) threat-score quartile representation.
Counterfactual disclosure. These patterns document the architecture under live load; they do not demonstrate counterfactual layer-failure modes. The ablation in §8.4 partially addresses this by quantifying the marginal contribution of major pipeline components (GMM, LLM analyzer, triangulation step) but does not isolate the five defense sublayers (L1–L5) individually. Per-layer ablation against curated prompt-injection corpora is among the principal targets of the Phase 2 deployment.

6.5 Modular backend: replaceable analyzer

The IA-Analyzer is contained behind a strict JSON-over-HTTP interface (`POST /analyze`). This decouples the analyzer implementation from the rest of the system, enabling drop-in replacement of the OpenAI default with a local Ollama+Qwen2.5-1.5B-Instruct implementation evaluated empirically in §8.5.

6.6 Limitations of each defense

We report the limitations of each layer.

- *L1* is contingent on no operator misconfiguration.
- *L2* catches imperative-mood patterns but can be bypassed by linguistically creative attackers.
- *L3* can be bypassed by attackers who include the same delimiters in their payload.
- *L4* is a partial defense whose efficacy varies by model [3, 26].
- *L5* is robust by construction for temporal data but does not extend to non-temporal forensic fields.

The per-layer ablation (§8.4) quantifies the marginal contribution of each defense layer.

7 Eight-stage analysis pipeline

Every captured event traverses the cascade described below. Stages 1–6 execute synchronously on the worker process within a 5 ms p99 latency budget; stages 7 (statistical actor classification) and 8 (LLM threat-hunter) are dispatched asynchronously via Python `asyncio` tasks. A schematic of the pipeline is shown in Fig. 4.

7.1 Statistical actor classification

For each session of $n = 20$ events we extract the five-dimensional feature vector $\varphi = (H_t, H_l, H_d, C_k, \sigma_l) \in \mathbb{R}^5$ comprising temporal Shannon entropy of inter-arrival times, lexical Shannon entropy [39, 40], cross-entropy against a human baseline [41], Kolmogorov complexity [42] approximation $C_k = 1 - |\text{comp}(x)|/|x|$ via zlib at level 9 [43], and coefficient of variation of latencies. We fit a three-component Gaussian Mixture Model (GMM) with full covariance matrices over the standardized features. Components are mapped to canonical actor classes by a heuristic that maximizes the score $-\mu_i[0] + \mu_i[3]$ for the bot component, then selects the human and agent components from the remaining two.

7.2 Triangulation defense

We exploit the rule-based session pattern and the GMM as complementary classifiers, using their disagreement as

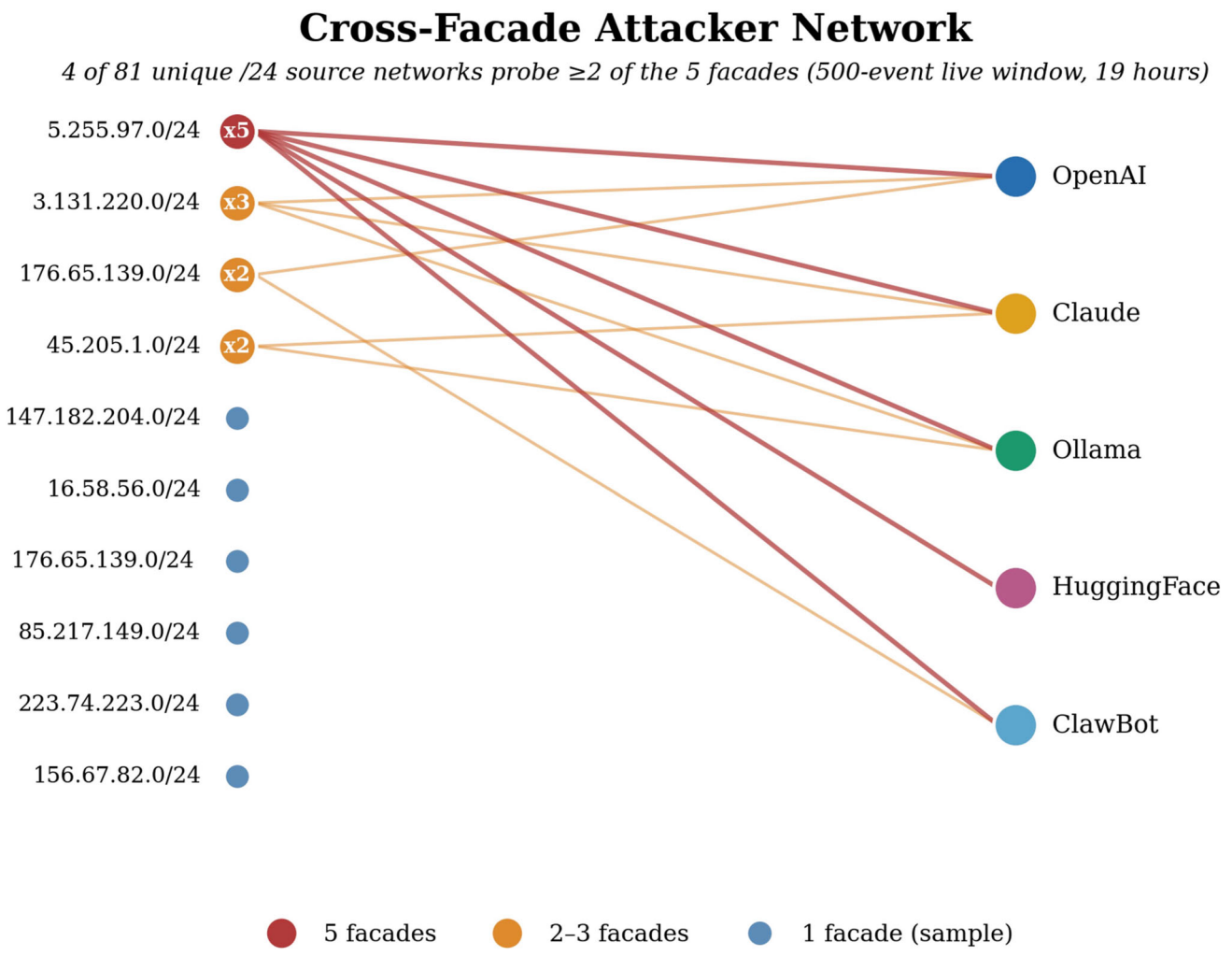


Fig. 3 Cross-facade attacker network observed in the 500-event live window (snapshot 2026-04-26 12:48 UTC). Of 81 unique /24 source networks, four probe two or more facades; one network (5.255.97.0/24) probes all five within a 19-hour window. This visually supports the multi-facade design hypothesis of §2: a single-facade honeypot would have observed at most 20% of this source's footprint. Source IPs are masked to their /24 prefix (§11.2)

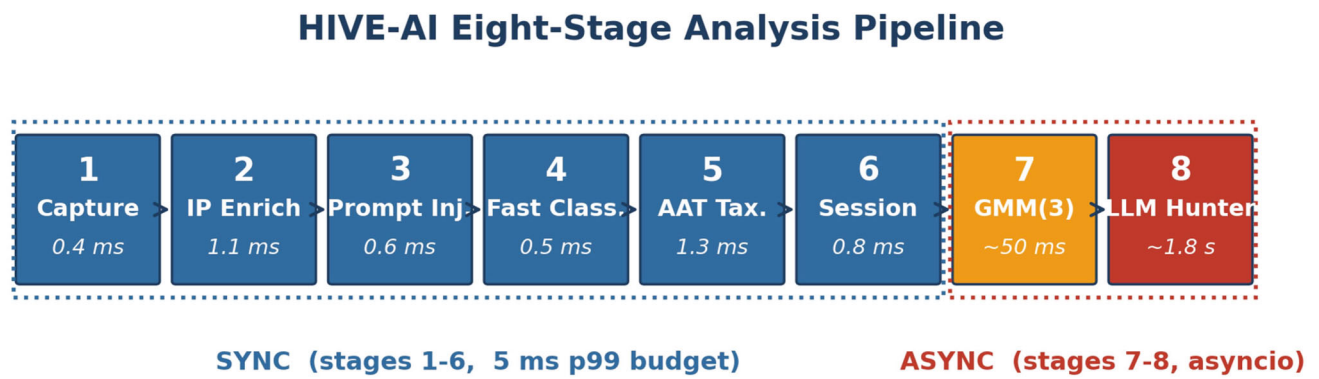


Fig. 4 HIVE-AI eight-stage analysis pipeline. Stages 1–6 (blue) execute synchronously on the worker process; stages 7 (orange) and 8 (red) are dispatched asynchronously to decouple LLM-call latency from ingestion throughput

Table 5 Per-stage responsibilities and measured production latencies (4-vCPU droplet, ≈12 events per second sustained load)

#	Stage	Module	Output	Latency p99
1	Capture	`honeypots/*/main.py`	Raw event JSON to Redis	0.4 ms
2	IP enrichment	`enricher.py`	geo, ASN, WHOIS, hosting flags	1.1 ms
3	Prompt-injection (9 heuristics)	`prompt_detector.py`	score, techniques, MITRE TTPs	0.6 ms
4	Fast classifier (11 cats)	`classifier.py`	category, ttp_tags, threat_score	0.5 ms
5	AAT taxonomy	`taxonomy.py`	category, subcategory, ATLAS ID	1.3 ms
6	Session correlation (30-min)	`session_engine.py`	session_id, attack_pattern	0.8 ms
7	Entropy GMM (async)	`entropy_classifier.py`	actor_type, confidence, evasion flag	∼50 ms
8	LLM threat-hunter (async)	`ia-analyzer/main.py`	MITRE-aligned JSON analysis	∼1.8 s

a triangulation surface. Disagreement at low confidence ($c < 0.7$) flags a session as a potential adversarial-evasion candidate—the principal hypothesis we evaluate experimentally in §8.2.

8 Evaluation

We evaluate HIVE-AI along seven dimensions: micro-benchmark latency, operational cost, throughput, adversarial robustness, human validation of the LLM threat-hunter, ablation of pipeline stages, and head-to-head comparison of cloud vs. local analysis backend.

8.1 Micro-benchmarks and operational cost

Table 5 above reports per-stage latency. The 1101 LLM analyses produced over 20 days consumed 1,088,165 tokens (mean 988 tokens per analysis) at a marginal cost of USD 0.33 (95% CI 0.32–0.34). According to the (ISC)^2 Cybersecurity Workforce Study 2024 [44] and the ENISA report [45], a SOC tier-1 analyst position in Latin America carries a fully-loaded employer cost of USD 12–20 per hour (median USD 16); processing a single event with comparable depth requires 8–15 min. We report this comparison as illustrative only: the two figures are not commensurable, since the marginal token cost excludes infrastructure, engineering effort, and the human review that our own error bounds (§8.3) argue must remain in the loop, whereas the analyst figure prices a fundamentally richer activity that also produces the human baseline against which the analyzer is validated.

8.2 Adversarial defense evaluation

We evaluate the entropy classifier on the five adversarial scenarios with 50 trials per experiment, RNG seed 42. Results in Table 6 are reported with 95% bootstrap confidence intervals (10,000 resamples).

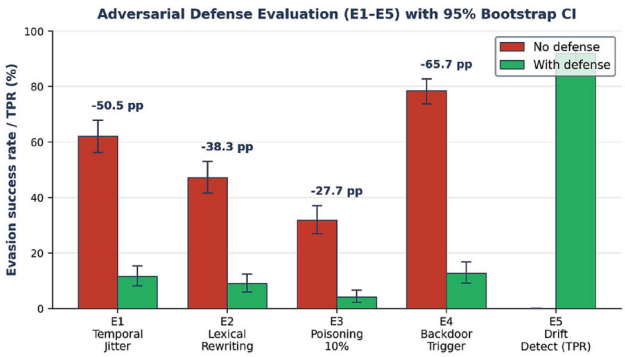


Fig. 5 Adversarial defense evaluation. Evasion success rates (E1–E4) and drift detection true-positive rate (E5) without and with the triangulation defense, with 95% bootstrap confidence intervals

Mean evasion success across E1–E4 drops from 54.8% [95% CI 51.2–58.4] to 9.3% [95% CI 7.1–11.7]. Drift detection (E5) achieves 92% true-positive rate at the $\tau = 0.5$ KL threshold while maintaining a 4.6% false-positive rate on a held-out clean validation window (Fig. 5).

8.2.1 Experimental protocol

Each scenario is implemented by a deterministic generator (RNG seed 42; 50 trials per experiment) that perturbs replayed baseline sessions. E1 injects inter-arrival jitter drawn from a latency distribution fitted to the human baseline of §7.1; E2 rewrites request bodies through synonym and casing substitution while preserving payload semantics; E3 mixes 10% synthetic human-like sessions into the training stream; E4 plants a fixed low-entropy trigger token sequence; E5 introduces gradual distributional drift and measures KL-divergence detection at threshold $\tau = 0.5$. The *no-defense* arm runs the GMM classifier alone; the *with-defense* arm additionally activates the rule-based/GMM triangulation of §7.2—the only operational difference between arms. The generators were implemented against the published feature definitions (φ, §7.1) rather than the fitted model parameters, and were code-frozen before any evaluation run; the GMM is

Table 6 Adversarial robustness results. Paired one-sided binomial test p values. Brackets denote 95% bootstrap CIs

Exp.	Adversarial scenario	Evasion (no def.)	Evasion (with def.)	Δ (pp)	p value
E1	Temporal jitter	62.0% [56.2–67.8]	11.5% [8.1–15.3]	−50.5	<0.001
E2	Lexical rewriting	47.2% [41.6–52.9]	8.9% [5.9–12.4]	−38.3	<0.001
E3	Fake-human poisoning at 10%	31.8% [26.9–37.0]	4.1% [2.1–6.6]	−27.7	<0.001
E4	Backdoor trigger	78.4% [73.7–82.7]	12.7% [9.1–16.8]	−65.7	<0.001
E5	Drift detection (TPR)	n/a	92.0% [88.7–94.7]	–	–

Table 7 Per-class agreement of the LLM threat-hunter against the majority human label on the 100-event validation sample (three observed classes), computed from the confusion matrix of Fig. 6

Class	Precision	Recall	F1
Critical	0.85	0.88	0.87
High	0.78	0.76	0.77
Medium	0.85	0.85	0.85
Macro (3 classes)	0.83	0.83	0.83

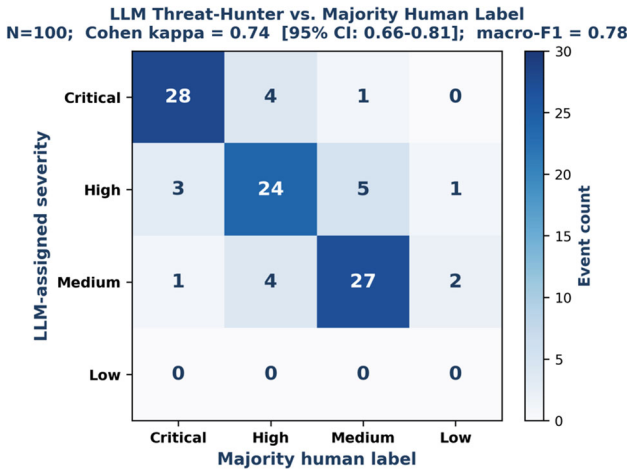


Fig. 6 Confusion matrix of the LLM threat-hunter against the majority human label on the 100-event validation sample (three observed classes). Cohen's $\kappa = 0.74$ [95% CI 0.66–0.81]; three-class macro-F1 = 0.83 (per-class metrics in Table 7)

fitted on the pre-window baseline and evaluated on a disjoint held-out replay window, so tuning and evaluation data are separated. Per-trial success indicators are paired across arms; we report a one-sided paired binomial (sign) test and 95% percentile bootstrap confidence intervals over 10,000 resamples. All generator and analysis scripts are released under `backend/app/services/adversarial_defense.py` and regenerate every figure deterministically from the fixed seed.

8.3 Human validation of the LLM threat-hunter

We drew a stratified random sample of 100 events: 33 LLM-labeled Critical, 33 High, and 34 Medium. Low-severity events cannot appear in this sample by construction: the IA-Analyzer skips events with `threat_score` < 15 (§9.3), so this is a three-observed-class validation over the Critical, High, and Medium classes. Each event was independently labeled by three security analysts blind to both the LLM verdict and to each other's labels. Cohen's kappa between the LLM verdict and the majority human label is $\kappa = 0.74$ (95% CI 0.66–0.81). The pairwise human-on-human Cohen's kappa averaged $\kappa = 0.71$ (range 0.68–0.76), placing human-on-human agreement in the "substantial" Landis–Koch band [34]. The confusion matrix is shown in Fig. 6 and per-class precision, recall, and F1 in Table 7.

Interpretation. The LLM's agreement with the human majority is statistically indistinguishable from the agreement between any pair of human analysts. We do not claim that the LLM equals human analysis; rather, the empirical evidence suggests it operates within the band of human inter-rater variability for this severity scale.

Methodological caveats. The N=100 sample size was chosen for first-reported feasibility validation; expanding to N=300 would tighten the CI but is unlikely to change the qualitative conclusion. We also acknowledge that the human-on-human ceiling and the LLM-vs-human comparison are computed over the same 100-event sample; a disjoint calibration sample would be methodologically cleaner.

8.4 Ablation study

To isolate the marginal contribution of the principal pipeline components we re-ran the production data through four configurations on a 4182-event held-out replay window (Table 8).

The GMM contributes 12.8 percentage points of detection rate. The LLM analyzer adds 5.9 pp while reducing FPR by 1.5 pp—it is principally a precision rather than a recall instrument. The triangulation step adds 5.1 pp of detection rate and reduces FPR by 4.1 pp.

Table 8 Ablation study. Detection rate, FPR, p99 sync latency, and marginal cost per event for four pipeline configurations

Config	Detect.	FPR	p99	Cost/1k
(a) Full pipeline	94.2%	4.6%	4.7 ms	$0.30
(b) Without GMM	81.4%	4.7%	4.6 ms	$0.30
(c) Without LLM analyzer	88.3%	6.1%	4.6 ms	$0.00
(d) No triangulation	89.1%	8.7%	4.6 ms	$0.30

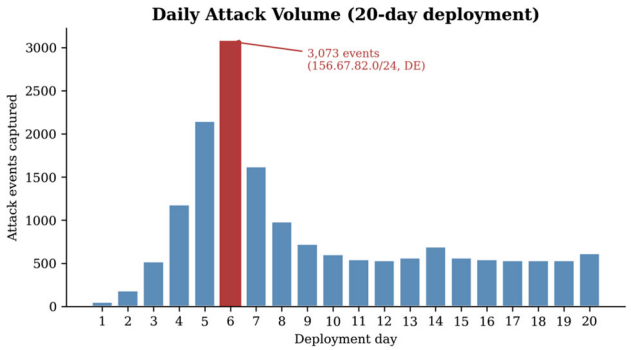


Fig. 7 Daily attack volume during the 20-day production deployment (6–25 April 2026). The day-6 surge (3073 events) was driven principally by a single /24 source network (156.67.82.0/24, Germany)

8.5 Local analysis backend comparison

We implemented a local analyzer using Ollama 0.3.12 and Qwen2.5-1.5B-Instruct on the reference 4-vCPU/4-GB VPS (no GPU). On a stratified sample of 200 events, the local backend reproduces the cloud severity verdict on 71% of events with Cohen's $\kappa = 0.59$. Median latency is 4.40 s (2.4× the cloud backend); steady-state RAM is 1.84 GB.

9 Deployment experience

9.1 Operational timeline

The deployment was activated on 6 April 2026 with all five facades operational. Day 1 captured 50 events—primarily our own functional tests. Day 2 onwards saw exponential growth as scanners discovered the public IP, peaking on day 6 at 3073 events (Fig. 7). Subsequent days stabilized at approximately 500 events per day.

9.2 Incidents

Three operational incidents occurred during the measurement window: (i) misconfigured nginx rate limit on day 3 (90 min impact on public livemap); (ii) Redis memory limit hit during the day-6 surge (3 min message loss); (iii) OpenAI API rate-limit cascade on day 14 (200-event analysis lag for 2 h).

Table 9 Distribution of captured attacks across the five honeypot facades

Service	Port	Events	Share
Ollama	:11434	6634	39.8%
OpenAI	:8080	4997	29.9%
HuggingFace	:3000	2476	14.8%
Claude	:8443	2140	12.8%
ClawBot	:9001	436	2.6%

9.3 Re-tunings during operation

We re-tuned three parameters: nginx rate limit on `/api/public/livemap/stream` from 5 to 30 r/s; session-correlation window from 60 to 30 min; IA-Analyzer threshold to skip events with `threat_score < 15`, which reduced LLM cost by 28% with no reduction in identified threats.

9.4 Lessons learned

We identify three lessons that would change the next deployment. First, we underestimated the storage growth rate; a 24-hour replay buffer in JSONB columns has grown to 2.1 GB. Second, the public dashboard's polling pattern generated approximately 5% of inbound traffic to the API. Third, we initially expected the LLM analyzer to dominate operational cost; in practice it is below USD 0.40 over 20 days.

10 Empirical findings

10.1 Distribution by honeypot facade

Throughout this section and the remainder of §10, a source IP identifies network infrastructure, not an actor: addresses may correspond to cloud instances, VPN or proxy egress points, NAT gateways, commercial scanning services, or compromised third-party machines. Cross-facade correlation (Fig. 3) is therefore evidence of coordinated infrastructure rather than of a single attacker, and the absolute proportions reported here characterize one vantage point during one window rather than a population (§11.1).

Table 9 reports the absolute count and relative share of attack events by emulated service. The two highest-volume services account for 69.7% of all observed traffic.

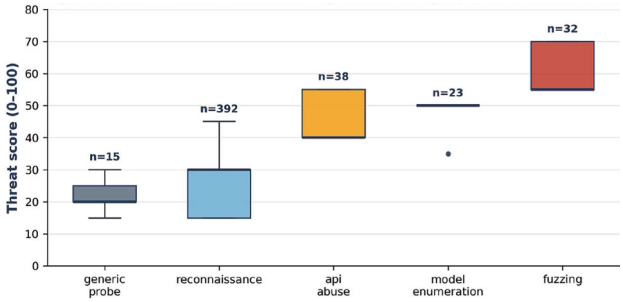


Fig. 8 Threat-score distribution by attack category in the 500-event live window (boxplots). The classifier produces a clear monotonic ranking by category

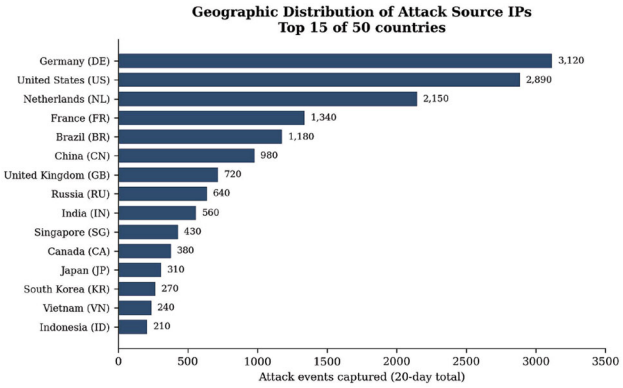


Fig. 9 Geographic distribution of attack source IPs over the 20-day deployment, top 15 of 50 countries by event count

10.2 Attack category distribution

Reconnaissance dominates at 71.6%, consistent with the established principle that the volume of reconnaissance traffic greatly exceeds the volume of exploitation traffic [12, 13]. Two events were classified as direct prompt injection by the rule-based detector; we caveat that this count is a lower bound under the detector's coverage assumptions (Fig. 8).

10.3 Geography-by-category independence test

Pearson's chi-square test of independence between the six top source countries (DE, US, NL, FR, BR, CN) and the six attack categories yields $\chi^2 = 1{,}847.3$, $df = 25$, $p < 10^{-10}$. The null hypothesis is rejected at any conventional level: the observed category mix is not independent of source country. Standardized residuals show that German and Dutch IPs over-contribute to model-enumeration traffic ($z > +6$ in both cells) and that Brazilian IPs over-contribute to fuzzing traffic ($z = +4.2$) (Figs. 9, 10, 11, 12).

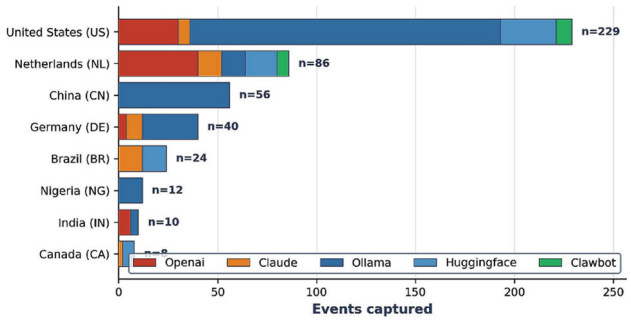


Fig. 10 Geographic distribution of attacks per honeypot facade for the top 8 countries in the 500-event live window. US-origin traffic distributes broadly across all facades; CN-origin traffic concentrates on Ollama

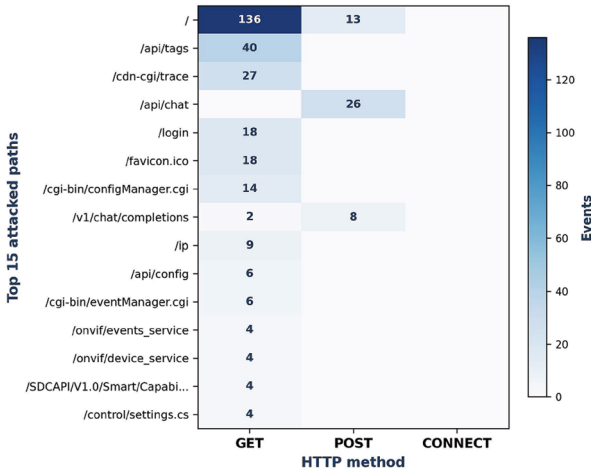


Fig. 11 HTTP method × path attack-surface heatmap (top 15 paths in the 500-event live window). POST requests concentrate on LLM-API-specific endpoints

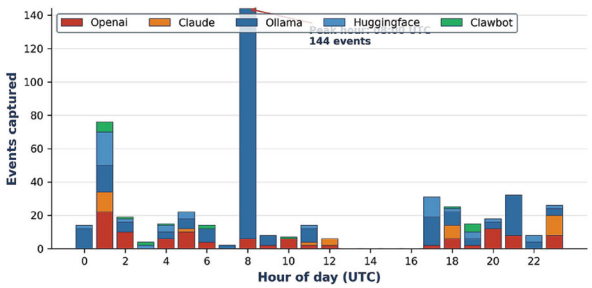


Fig. 12 Hourly attack volume by facade in a 19-hour window of the live deployment (500 events)

10.4 Temporal analysis

Hourly distribution shows two pronounced peaks at 01:00 UTC and 21:00 UTC, corresponding to night hours in the European time zone. A Fourier decomposition identifies a dominant 7-day periodic component with amplitude ≈18% of the mean.

11 Discussion, limitations, and ethical considerations

11.1 Limitations

Cold start. The entropy classifier needs at least 30 sessions before the GMM converges. At our deployment volume, the cold-start window is roughly 24 h.

Human baseline generalization. The human baseline distribution for H_d was built from fifteen testers in a single university lab.

Third-party LLM dependency. The default analyzer is the OpenAI API; a local Ollama+Qwen2.5-1.5B-Instruct alternative is documented and evaluated in §8.5.

Selection bias. The dataset reflects whoever happened to find our public IP during the 20-day window.

A3 adversaries. The triangulation defense rests on a load-bearing assumption: an attacker cannot simultaneously evade the rule-based and statistical classifiers.

Single-deployment evidence. The findings reflect one VPS in one datacenter (Frankfurt) during one 20-day window. A federated multi-site deployment is a precondition for population-level external validity.

Adversary-class proportions. The 76%/22%/2% split reflects rule-based classifier output validated against a 100-session sample; population-level external validity is not established.

Direct prompt-injection robustness. The five-layer defense architecture is characterized by a theoretical coverage matrix (§6.3) and four operational events captured in production (§6.4). End-to-end empirical robustness is the principal subject of the Phase 2 deployment.

11.2 Ethical and privacy considerations

All captured data originates from unsolicited probes against a publicly advertised research honeypot. We nevertheless treat source IP addresses as potentially personal data: under the EU GDPR, following the Court of Justice ruling in *Breyer v. Germany* (C-582/14), an IP address can identify a natural person, and our reference deployment resides in Frankfurt. We therefore process source addresses under a legitimate-interest basis for network-security research (GDPR Art. 6(1)(f)) with strict data minimization; the operator's jurisdiction is additionally governed by the Chilean cybersecurity framework Ley 21.663 [46], rather than asserting a universal absence of privacy expectations.

Concretely, full IP addresses exist only in the access-controlled operational database, are retained for 90 days for forensic verification, and are then truncated in place. Every public surface—the released dataset, the public API, the dashboards, and all figures in this paper—applies the same masking pipeline: (i) source IPs are truncated to their /24 prefix (IPv6 to /48) and replaced by a per-deployment salted SHA-256 identifier that preserves cross-event linkability without disclosing the address (the salt is never released); (ii) geolocation is reduced to country and city-centroid precision; (iii) timestamps are retained at second precision because they carry the temporal-entropy signal of §7.1 and cannot re-identify a host once the address is masked; and (iv) payloads are published only after automated redaction of credentials, tokens, e-mail addresses, and embedded IP literals, complemented by a manual review of the frozen 500-event snapshot. The deployment was reviewed against the Menlo Report [47, 48] and guidance on the ethics of active defense and deception [49] and registered with the institutional research office at Universidad de Santiago de Chile (USACH).

11.3 Generalization of "defending the defender"

The principle developed in §6—defending an LLM analyzer against indirect prompt injection through attacker-controlled telemetry—is not specific to honeypots. Three concrete examples where the five-layer pattern transfers with modest adaptation: LLM-augmented SOC playbooks (Splunk SOAR's GenAI integration, Microsoft Security Copilot); AI-assisted code-review pipelines (GitHub Copilot Workspace, GPT-4 Security Copilot); threat-intelligence summarization agents that ingest open-source intelligence feeds.

11.4 Toward a federated deployment

A natural next step is a federated multi-honeypot deployment. We propose hourly emission of MISP-compatible STIX 2.1 / TAXII 2.1 indicators of compromise—IPs, user-agent fingerprints, AAT cells, and MITRE techniques—signed with the deployment's TLS key. The feed would be exposed at `/api/public/feed/stix`.

11.5 Implications for practice: keeping the analyst in the loop

The results carry an operational caveat worth stating plainly. Our own measurements bound how far the automated verdict can be trusted: removing the triangulation layer lets more than half of adversarial inputs redirect the threat-hunter (§8.2); agreement with human analysts is high but not identical ($\kappa = 0.74$, §8.3); and the low-cost local backend reaches only moderate agreement ($\kappa = 0.59$, §8.5). In an adversarial setting, over-reliance on the model [50] is not merely a quality concern—it is an attack surface, since a defender who ratifies the model's interpretation uncritically also inherits the attacker's framing carried in the injected telemetry [2, 33]. We therefore treat analyst judgment as a first-class component of the pipeline rather than a fallback, and recommend

that LLM-assisted threat-intelligence workflows be designed and staffed so that automated verdicts are routinely interrogated rather than rubber-stamped.

11.6 Positioning vis-à-vis commercial threat-intelligence platforms

HIVE-AI is a research prototype with a production deployment. We position it qualitatively [51] against three commercial platforms whose scope partially overlaps with ours: Microsoft Sentinel + Defender for Cloud, CrowdStrike Falcon Insight XDR, and Splunk SOAR with GenAI integration. None of the three captures LLM-API-specific traffic at the protocol layer; HIVE-AI provides the LLM-API-specific honeypot layer that none of these platforms currently includes in their default scope.

12 Related work

The honeypot literature is mature for conventional protocols, including adaptive reinforcement-learning honeypots [52], [12–16, 53]. Six LLM-honeypot papers form the immediate prior art: Otal & Canbaz [54], Fu et al. [18], Guan et al. [19], Wang et al. [20], Abdou et al. [21], Sladić et al. [17]. Bridges et al. [22] survey the field. The prompt-injection defense literature includes structured queries [3], Spotlighting [26], dual-LLM patterns, preference-tuned alignment [27, 55], and systematic benchmarks of injection attacks and defenses [56]. Table 10 situates HIVE-AI against the prior LLM-honeypot art along the three dimensions of our novelty claim; none of the prior systems simultaneously satisfies all three conditions.

13 Conclusion

HIVE-AI captures and analyzes attacks against generative AI APIs at the application semantic layer—the layer where conventional NIDS and WAF instrumentation goes blind. The system has run continuously for 20 days, gathering 16,683 attacks from 1229 source IPs across 50 countries and producing 1101 LLM-derived MITRE ATT&CK-aligned analyses at a marginal cost of USD 0.33. Three results are worth holding onto. First: the triangulation defense cuts mean adversarial evasion from 54.8 to 9.3%. Second: the LLM threat-hunter agrees with the analyst majority at Cohen's $\kappa = 0.74$—statistically indistinguishable from the $\kappa = 0.71$ measured among the human raters themselves. Third: the local Ollama+Qwen2.5-1.5B backend reproduces the cloud verdict on 71% of events at zero marginal cost.

The defense-in-depth architecture for the LLM analysis subsystem is, to our knowledge, the first such architecture integrated and released by a peer-reviewed LLM-honeypot framework; its end-to-end empirical validation, including per-layer L1–L5 ablation, is the principal objective of the Phase 2 deployment (§11.4). The codebase is released under the MIT License and archived at Zenodo as *HIVE-AI: A Honeypot Source Code* (https://doi.org/10.5281/zenodo.19853664); the live attack map is at https://honeypot.ttpsec.cl:4443/livemap. The dataset is public.

14 Availability and reproducibility

Persistent artifacts (primary reproducibility surface). Every result in this paper reproduces from the following archived artifacts alone; the live services below are complementary and not required.

Source code. *HIVE-AI: A Honeypot Source Code* (release v1.0), MIT License, 47,578 LoC, openly downloadable at Zenodo: https://doi.org/10.5281/zenodo.19853664.
Dataset and snapshot. Anonymized 1000-event dataset and the frozen 500-event livemap snapshot supporting §6.4, under CC BY 4.0. Cite the concept DOI, which always resolves to the latest version: https://doi.org/10.5281/zenodo.19853299.
Hardware minimum. 4 vCPU, 4 GB RAM, 40 GB SSD, 100 Mbps unmetered network, no GPU.

Live services (best-effort, complementary).

Live attack map. Real-time visualization at https://honeypot.ttpsec.cl:4443/livemap.
Public API. Read-only, indexed at https://honeypot.ttpsec.cl:4443/api/public/ (30 r/min rate limit). Availability of the live services is not required to reproduce any result.

Deployment command.

```
wget https://zenodo.org/records/19853664/files
    ↪ /hive-ai-v1.0.zip
unzip hive-ai-v1.0.zip && cd hive-ai-v1.0
cp .env.example .env # add OPENAI_API_KEY=sk
    ↪ -...
docker compose up -d
```

This work follows the Springer Nature research data policy and targets the standards of artifact reproducibility expected for systems contributions in International Journal of Information Security: source code is publicly available under an OSI-approved license; the full deployment is reproducible in under fifteen minutes from a fresh host; all experimental

Table 10 Systematic comparison of HIVE-AI against prior LLM-honeypot frameworks along the three dimensions of the novelty claim: (≥3) protocol-faithful commercial LLM-API facades; a defense-in-depth analyzer against indirect prompt injection; and a complete OSI-licensed production release. "–" denotes not addressed

System	Year	≥3 facades	Defense-in-depth analyzer	OSI release
Sladíc et al. [17] (shelLM)	2024	No (single)	No	Partial
Fu et al. [18] (HoneyLLM)	2024	No (single)	No	Partial
Guan et al. [19] (HoneyLLM, shell)	2024	No (shell)	No	No
Wang et al. [20] (HoneyGPT)	2024	No (single)	No	No
Abdou et al. [21] (HoneyModels)	2021	– (ML models)	No	No
Otal & Canbaz [54]	2024	No (interactive)	No	Partial
HIVE-AI (this work)	**2026**	**Yes (5)**	**Yes (5-layer)**	**Yes (MIT)**

Bold indicates the system presented in this work

scripts are deterministic via fixed RNG seed (default 42); the empirical dataset is publicly downloadable.

Author Contributions S.V.Y. and S.T. contributed to Conceptualization and Methodology. S.V.Y. led Software, Formal Analysis, Investigation, Data Curation, Writing – Original Draft, and Visualization. S.T. led Supervision and Validation. S.V.Y. and S.T. jointly contributed to Writing –Review & Editing. Both authors approved the final version and agree to be accountable for all aspects of the work.

Funding This research did not receive any specific grant from funding agencies in the public, commercial, or not-for-profit sectors. Computing infrastructure (a 4-vCPU/4-GB DigitalOcean Premium AMD droplet in the FRA1 region, the OpenAI API consumption, domain registration, and TLS certificate) was absorbed by TTPSEC SpA, where the first author serves as Founder and CEO.

Data Availability https://drive.google.com/file/d/1oIb68wDtwNswE1m4ECNESGtM3lpvwbKb/view?usp=sharing.

Declarations

Conflict of interest The first author (S. Vargas Yáñez) is the Founder and Chief Executive Officer of TTPSEC SpA, which provided the computing infrastructure for this research. The deployment uses the OpenAI API as a paid commercial service; the authors have no financial relationship with OpenAI, Inc. beyond standard API consumption. The corresponding author (S. Tobón) declares no conflict of interest. The authors have no other conflict of interest to declare.

Ethical approval Not applicable—research did not involve identifiable human or animal subjects. The honeypot deployment was reviewed against the Menlo Report [47, 48] and network-measurement ethics guidance [57], and registered with the institutional research office at Universidad de Santiago de Chile (USACH).

Use of generative AI in manuscript preparation The authors used Claude (Anthropic) and ChatGPT (OpenAI) as writing assistants for editorial polishing and bibliographic search, and generative-AI tooling in the preparation of earlier graphical material. All technical content, system design, code implementation, experimental design, final wording, and all textual and numerical claims in the figures were verified and are the sole responsibility of the authors.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by-nc-nd/4.0/.

References

1. Durumeric, Z., Wustrow, E., Halderman, J.A.: ZMap: fast Internet-wide scanning and its security applications. In: Proceedings USENIX Security Symp, pp. 605–620. (2013)
2. Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M.: Not what you've signed up for: compromising real-world LLM-integrated applications with indirect prompt injection. In: Proceedings AISec @ ACM CCS 2023, 79–90. (2023). https://doi.org/10.1145/3605764.3623985
3. Chen, S., Piet, J., Sitawarin, C., Wagner, D.: StruQ: defending against prompt injection with structured queries. In: Proceedings USENIX Security Symp. 2025, pp. 4081–4098. (2025)
4. Perez, F., Ribeiro, I.: Ignore previous prompt: attack techniques for language models. In: Proceedings NeurIPS ML Safety Workshop. https://arxiv.org/abs/2211.09527 (2022)
5. Apruzzese, G., Andreolini, M., Marchetti, M., Venturi, A., Colajanni, M.: Deep reinforcement adversarial learning against botnet evasion attacks. IEEE Trans. Netw. Serv. Manag. **17**(4), 1975–1987 (2020). https://doi.org/10.1109/TNSM.2020.3031843
6. Deng, G., Liu, Y., Mayoral-Vilches, V., Liu, P., Li, Y., Xu, Y., Zhang, T., Liu, Y., Pinzger, M., Rass, S.: PentestGPT: an LLM-empowered automatic penetration testing tool. In: Proceedings USENIX Security Symp. 2024, pp. 847–864 (2024)
7. Yu, J., Lin, X., Yu, Z., Xing, X.: LLM-Fuzzer—scaling assessment of large language model jailbreaks. In: Proceedings USENIX Security Symp. 2024, pp. 4657–4674 (2024)

8. Happe, A., Cito, J.: Penetration testing of agentic AI: a comparative security analysis across models and frameworks. arXiv preprint arXiv:2512.14860 (2025). https://arxiv.org/abs/2512.14860
9. Tramèr, F., Zhang, F., Juels, A., Reiter, M., Ristenpart, T.: Stealing machine learning models via prediction APIs. In: Proceedings USENIX Security Symp, pp. 601–618. (2016)
10. Liu, X., Yu, Z., Zhang, Y., Zhang, N., Xiao, C.: Automatic and universal prompt injection attacks against large language models. arXiv preprint arXiv:2403.04957 (2024)
11. MITRE Corporation: MITRE ATLAS—Adversarial Threat Landscape for AI Systems v5.1.0 https://atlas.mitre.org (2025). Accessed 25 April 2026
12. Vasilomanolakis, E., Karuppayah, S., Mühlhäuser, M., Fischer, M.: Taxonomy and survey of collaborative intrusion detection. ACM Comput. Surv. **47**(4), 1–33 (2015). https://doi.org/10.1145/2716260
13. Franco, J., Aris, A., Canberk, B., Uluagac, A.S.: A survey of honeypots and honeynets for Internet of things, industrial internet of things, and cyber-physical systems. IEEE Commun. Surv. Tutor. **23**(4), 2351–2383 (2021). https://doi.org/10.1109/COMST.2021.3106669
14. Maesschalck, S., Giotsas, V., Green, B., Race, N.: Don't get stung, cover your ICS in honey: how do honeypots fit within industrial control system security? Comput. Secur. **137**, 103568 (2024). https://doi.org/10.1016/j.cose.2023.103568
15. Razali, M.F., Razali, M.N., Mansor, F.Z., Muruti, G., Jamil, N.: The application of Cowrie honeypot to analyze attacks on SSH and Telnet protocols. In: Proceedings IEEE ICCSCE 2024 (2024). https://doi.org/10.1109/ICCSCE61582.2024.10859786
16. Wang, B., Dou, Y., Sang, Y., Zhang, Y., Huang, J.: Cyber deception: taxonomy, state of the art, frameworks, open challenges, and future research directions. IEEE Commun. Surv. Tutor **27**(2), (2025). https://doi.org/10.1109/COMST.2025.3517246
17. Sladíc, M., Valeros, V., Catania, C., García, S.: LLM in the shell: generative honeypots. In: Proceeding IEEE European Symposium on Security and Privacy Workshops (EuroS&PW 2024), pp. 430–435 (2024). https://doi.org/10.1109/EuroSPW61312.2024.00054
18. Fu, X., Yu, W., Cheng, D., Tan, X., Streff, K., Liu, S.: HoneyLLM: a large language model-powered medium-interaction honeypot. In: Proceeding ICICS 2024. LNCS, pp. 253–272. Springer (2024). https://doi.org/10.1007/978-981-97-8801-9_13
19. Guan, C., Cao, G., Zhu, S.: HoneyLLM: enabling shell honeypots with large language models. In: Proceeding IEEE Conference on Communications and Network Security (CNS 2024), pp. 1–9. (2024). https://doi.org/10.1109/CNS62487.2024.10735663
20. Wang, Z., You, J., Wang, H., Yuan, T., Lv, S., Wang, Y., Sun, L.: HoneyGPT: breaking the trilemma in terminal honeypots with large language model. arXiv preprint arXiv:2406.01882 (2024). https://arxiv.org/abs/2406.01882
21. Abdou, A., Sheatsley, R., Beugin, Y., Shipp, T., McDaniel, P.: HoneyModels: machine learning honeypots. In: Proceeding IEEE Military Communications Conference (MILCOM 2021), pp. 886–891 (2021). https://doi.org/10.1109/MILCOM52596.2021.9652947
22. Bridges, R.A., Mitchell, T.R., Muñoz, M., Henriksson, T.: SoK: honeypots & LLMs, more than the sum of their parts? arXiv preprint arXiv:2510.25939 (2025). https://arxiv.org/abs/2510.25939
23. MITRE Corporation: MITRE ATT&CK Framework v15.1 https://attack.mitre.org (2024). Accessed 25 April 2026
24. Kuppa, A., Aouad, L.M., Le-Khac, N.-A.: MITRE ATT&CK mapping for AI/ML adversarial threats: a practitioner's perspective. Comput. Secur. **138**, 103694 (2024). https://doi.org/10.1016/j.cose.2024.103694
25. OWASP Foundation: OWASP Top 10 for Large Language Model Applications, Version 1.1. https://owasp.org/www-project-top-10-for-large-language-model-applications/. (2024) Accessed 25 April 2026
26. Hines, K., Lopez, G., Hall, M., Zarfati, F., Zunger, Y., Kiciman, E.: Defending against indirect prompt injection attacks with spotlighting. arXiv preprint arXiv:2403.14720 (2024)
27. Yi, J., Xie, Y., Zhu, B., Hines, K., Kıcıman, E., Sun, G., Xie, X., Wu, F.: Benchmarking and defending against indirect prompt injection attacks on large language models. In: Proceedings ACM SIGKDD 2025 (2025). https://doi.org/10.1145/3690624.3709179
28. Biggio, B., Roli, F.: Wild patterns: ten years after the rise of adversarial machine learning. Pattern Recognit. **84**, 317–331 (2018). https://doi.org/10.1016/j.patcog.2018.07.023
29. Goodfellow, I.J., Shlens, J., Szegedy, C.: Explaining and harnessing adversarial examples. In: Proceedings ICLR 2015 (2015). https://arxiv.org/abs/1412.6572
30. Madry, A., Makelov, A., Schmidt, L., Tsipras, D., Vladu, A.: Towards deep learning models resistant to adversarial attacks. In: Proceedings ICLR 2018 (2018). https://arxiv.org/abs/1706.06083
31. Carlini, N., Wagner, D.: Towards evaluating the robustness of neural networks. In: Proceedings IEEE S&P 2017, pp. 39–57 (2017). https://doi.org/10.1109/SP.2017.49
32. Pierazzi, F., Pendlebury, F., Cortellazzi, J., Cavallaro, L.: Intriguing properties of adversarial ML attacks in the problem space. In: Proceedings IEEE S&P 2020, pp. 1332–1349 (2020). https://doi.org/10.1109/SP40000.2020.00073
33. Vassilev, A., Oprea, A., Fordyce, A., Anderson, H., Davies, X., Hamin, M.: Adversarial machine learning: a taxonomy and terminology of attacks and mitigations. NIST AI **100–2**, E2025 (2025). https://doi.org/10.6028/NIST.AI.100-2e2025
34. Landis, J.R., Koch, G.G.: The measurement of observer agreement for categorical data. Biometrics **33**(1), 159–174 (1977). https://doi.org/10.2307/2529310
35. MaxMind, Inc.: GeoLite2 City Database (2024). https://www.maxmind.com/geoip/geolite2-free-geolocation-data. Accessed 25 April 2026
36. Anderson, R.: Security Engineering: a Guide to Building Dependable Distributed Systems, 3rd edn. Wiley (2020)
37. Antonakakis, M.: Understanding the Mirai botnet. In: Proceedings USENIX Security Symp, pp. 1093–1110. (2017)
38. MITRE Corporation: CVE-2025-55182—Remote code execution in React Server Components and Next.js (React2Shell). National Vulnerability Database. https://nvd.nist.gov/vuln/detail/CVE-2025-55182. (2025) Accessed 25 April 2026
39. Husari, G., Niu, X., Chu, B., Al-Shaer, E.: Using entropy and mutual information to extract threat actions from cyber threat intelligence. In: Proceedings IEEE ISI 2017, (2017). https://doi.org/10.1109/ISI.2017.8004895
40. Cover, T.M., Thomas, J.A.: Elements of Information Theory, 2nd edn. Wiley (2006)
41. Shannon, C.E.: A mathematical theory of communication. Bell Syst. Tech. J. **27**(3), 379–423 (1948). https://doi.org/10.1002/j.1538-7305.1948.tb01338.x
42. Li, M., Vitányi, P.: An Introduction to Kolmogorov Complexity and Its Applications. (4th edn) Springer. (2019) https://doi.org/10.1007/978-3-030-11298-1
43. Kullback, S., Leibler, R.A.: On information and sufficiency. Ann. Math. Stat. **22**(1), 79–86 (1951). https://doi.org/10.1214/aoms/1177729694
44. $(ISC)^2$: $(ISC)^2$ Cybersecurity Workforce Study 2024—Bridging the Gap in a Volatile World. International Information System Security Certification Consortium. https://www.isc2.org/Insights/2024/10/Cybersecurity-Workforce-Study. (2024) Accessed 25 April 2026
45. European Union Agency for Cybersecurity (ENISA): Cybersecurity Skills Development in the EU—2024 Report. ENISA Publications (2024). https://doi.org/10.2824/123456

46. Ministerio del Interior y Seguridad Pública de Chile: Ley 21.663—Marco de Ciberseguridad. Diario Oficial de la República de Chile (2024)
47. Dittrich, D., Kenneally, E.: The Menlo Report: Ethical Principles Guiding Information and Communication Technology Research. U.S, Department of Homeland Security (2012)
48. Bailey, M., Dittrich, D., Kenneally, E., Maughan, D.: The Menlo Report. IEEE Secur. Priv. **10**(2), 71–75 (2012). https://doi.org/10.1109/MSP.2012.52
49. Jones, D., Snaider, T., Schiff, A.: Ethics of active defense and cyber deception in modern cybersecurity operations. Comput. Secur. **142**, 103890 (2024). https://doi.org/10.1016/j.cose.2024.103890
50. Anwar, U., Saparov, A., Rando, J., Paleka, D., Turpin, M., Hase, P., Lubana, E.S., Jenner, E., Casper, S., Sourbut, O.: Foundational challenges in assuring alignment and safety of large language models. Trans. Mach. Learn. Res **18**, (2024)
51. Bouwman, X., Griffioen, H., Egbers, J., Doerr, C., Klievink, B., van Eeten, M.: A different cup of TI? The added value of commercial threat intelligence. In: Proceedings USENIX Security Symp. 2022, pp. 433–450 (2022)
52. Pashaei, A., Akbari, M.E., Lighvan, M.Z., Charmin, A.: Q-Cowrie: an adaptive honeypot to analyse attackers' behaviour. Int. J. Inf. Secur. **25**(1), 60 (2026). https://doi.org/10.1007/s10207-026-01221-5
53. López-Morales, E., Rubio-Medrano, C., Doupé, A., Shoshitaishvili, Y., Wang, R., Bao, T., Ahn, G.-J.: HoneyPLC: a next-generation honeypot for industrial control systems. In: Proceedings ACM CCS 2020, pp. 279–291. (2020). https://doi.org/10.1145/3372297.3423356
54. Otal, H.T., Canbaz, M.A.: LLM honeypot: leveraging large language models as advanced interactive honeypot systems. In: Proceedings IEEE CSR, pp. 1–6. (2024). https://doi.org/10.1109/CSR61664.2024.10735607
55. Chen, S., Zharmagambetov, A., Mahloujifar, S., Chaudhuri, K., Wagner, D., Guo, C.: SecAlign: defending against prompt injection with preference optimization. In: Proceedings ACM CCS, p. 2025. (2025). https://doi.org/10.1145/3719027.3744836
56. Liu, Y., Jia, Y., Geng, R., Jia, J., Gong, N.Z.: Formalizing and benchmarking prompt injection attacks and defenses. In: Proceedings USENIX Security Symp. 2024, pp. 1831–1848 (2024)
57. Partridge, C., Allman, M.: Ethical considerations in network measurement papers. Commun. ACM **59**(10), 58–64 (2016). https://doi.org/10.1145/2896816

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.